Contents

1 Specifications Miscellaneous Autolisp Tools

1.1 Format

(format control-string (list arguments...)) -> string

control-string is a subset of the Common Lisp format control string syntax.

The generic syntax for a control string specifier is:

```
control-string-specifier ::= "~" [ [parameter]  { "," [parameter] } ] { "@" | ":" } spe
parameter ::= integer | "V" | "'" character .
specifier ::= letter | "%" |  "&" | "~" | "$"
            | "{" | "}" | "?" | "[" | "]" | ";" | "^" | "*"
            | "<" | ">" | "_" | "/" identifier "/" | newline .
```

specifier letters and "V" in parameter are case insensitive. (But the character after "'" in parameter is case sensitive (preserved)). identifier is case insensitive.

You may implement this syntax that cover the whole gamut for the parsing of the specifiers, but in the semantic phase, specifiers and parameters not specified in the following table should signal an error.

Syntax	Meaning	Notes
~A	vl-princ-to-string	any object
~wA	" w: integer, total width	" ~A left-aligns and pads with spaces; width may
~VA	"	" takes an integer argument (before the object) f
~S	vl-prin1-to-string	any object
~wS	" w: integer, total width	" ~S left-aligns and pads with spaces; width may
~VS	"	" takes an integer argument (before the object) f
~D	itoa	takes a number, convert to int
~wD	" w: integer, total width	" ~D right-aligns and pads with spaces; width m
~B	print integer argument in binary	takes a number, convert to int
~wB	" w: integer, total width	" ~B right-aligns and pads with spaces; width m
~O	print integer argument in octal	takes a number, convert to int
~wO	" w: integer, total width	" ~O right-aligns and pads with spaces; width m
~X	print integer argument in hexadecimal	takes a number, convert to int
~wX	" w: integer, total width	" ~X right-aligns and pads with spaces; width m
~F	rtos	takes a number, convert to float
~W,dF	" w: integer, total width	"
	" d: integer, number of decimals	"
~:F	angtos	takes a number, convert to float
~W,d:F	" w: integer, total width	"
	" d: integer, number of decimals	"
~C	prints a single character	takes only a 1-character string. If string length>
~@C	prints a single character prefixed by #\	takes only a 1-character string. If string length>
~%	terpri	
~n%	" n: integer, number of newlines	
~V%	"	takes an integer argument for the number of new
~&	terpri	actually outputs a newline only if we're not at co
~n&	" n: integer, number of newlines	actually outputs n-1 newlines if we're already at
~V&	"	takes an integer argument for the number of new
~~	(princ "~")	
n	n: integer, number of tildes	n times (princ "~")
V		takes an integer argument for the number of tild
n{s}	iteration	takes one argument that should be a list of argu
n:{s}	iteration	takes one argument that should be a list of list o
n@{s}	iteration	takes arguments (possibly several per iteration as
n:@{s}	iteration	takes at most n arguments that are list of argum

More details if needed can be seen at https://www.lispworks.com/

`documentation/HyperSpec/Body/22_c.htm` and subpages. (But not the full CL format specifier syntax and semantics need to be implemented, only what is specified in this table.

1.1.1 padding

~D ~B ~O ~X may take a second parameter, a padchar. ~A ~S take additionnal parameters, the padchar being the 4th (parameters are actually optional between commas).

(and (equal (format "~10,,,'-A" '("hello")) "hello-----") (equal (format "~10,,,'-S" '("hello")) "\"hello\"-----") (equal (format "~10,'-D" '(42)) "-------42") (equal (format "~10,'#D" '(42)) "########42") (equal (format "~10,'#O" '(42)) "########52") (equal (format "~10,'#X" '(42)) "########2A") (equal (format "~16,'#B" '(42)) "##########101010") (equal (format "~16,'#B" '(-42)) "#########-101010") (equal (format "~16,'#X" '(-42)) "############-2A") (equal (format "~16,'#O" '(-42)) "############-52") (equal (format "~16,'#D" '(-42)) "############-42"))

1.1.2 Example of expected results

```
(progn
  (princ (format "~~A    ~A<~%~~10A  ~10A<~%~~VA   ~VA<~%~~S    ~S<~%~~10S  ~10S<~%~~VS
                 '("Hello" "World" 20 "Hello World!"
                   "Hello" "World" 20 "Hello World!")))
 (princ (format "  {~%~{| ~10A | ~10D~%~}~%" '(("apple" 12 "orange" 423 "strawberry" 12
 (princ (format " :{~%~:{| ~10A | ~10D~%~}~%" '((("apple" 12 ripe) ("orange" 423 green)
 (princ (format " @{~%~@{| ~10A | ~10D~%~}~%" '("apple" 12 "orange" 423 "strawberry" 12
 (princ (format ":@{~%~:@{| ~10A | ~10D~%~}" '(("apple" 12 ripe) ("orange" 423 green)

autolisp> (format "~{| ~10A | ~10D~%~}" '(("apple" 12 "orange" 423 "strawberry" 1200))
"| apple      |         12
| orange     |        423
| strawberry |       1200
"
autolisp> (format "~:{| ~10A | ~10D~%~}" '((("apple" 12 ripe) ("orange" 423 green) ("st
"| apple      |         12
| orange     |        423
| strawberry |       1200
```

```
"
autolisp> (format "~@{| ~10A | ~10D~%~}" '("apple" 12 "orange" 423 "strawberry" 1200))
"| apple      |         12
| orange     |        423
| strawberry |       1200
"
autolisp> (format "~:@{| ~10A | ~10D~%~}" '(("apple" 12 ripe) ("orange" 423 green) ("st
"| apple      |         12
| orange     |        423
| strawberry |       1200
"
```